6.15 Unique (Lab 6.2)

LAB ACTIVITY | 6.15.1: Unique (Lab 6.2) | Full screen | 0 / 30

The well-known children's book *Green Eggs and Ham* by T. S. Geisel is famous for only using 50 words (51 if you count the proper name "Sam-i-am"). This statistic can be computed as well, as shown in this exercise.

Download this template and rename it `unique.py`:

- template.py

Write a function named `get_unique_words` that collects and returns the *unique* words in the text. This problem requires that you keep track of the words you've seen so far. The following algorithm implements this behavior:

1. **Given**: a list of strings, called `str_list`.
2. **Create** an empty list called `unique_words`
3. **Repeat** for each word in `str_list`
    1. **If** the current word is not already in `unique_words` **then**
        1. Add the current word to unique_words.
4. **Return** `unique_words`.

Notice a couple things about this algorithm before implementing it:

- It uses an *accumulator* (`unique_words`) to keep track of what it has seen so far as it works through the list.
- It handles an empty list gracefully by returning `[]`.
- It returns a list of words, which can be printed or counted.

Document your function in the standard way; you do not need to include the algorithm. Note the following about the implementation:

- To implement the search step (i.e., "the current word is not already in `uniqueWords`"), copy your search function from the previous lab exercise into this module and use it to check to see if the current word is not already in the list (i.e., by saying `search(unique_words, current_word) == -1`).
- To implement the adding of a new unique word to `unique_words`, remember that you can append items to a list using `my_list.append(new_item)`.

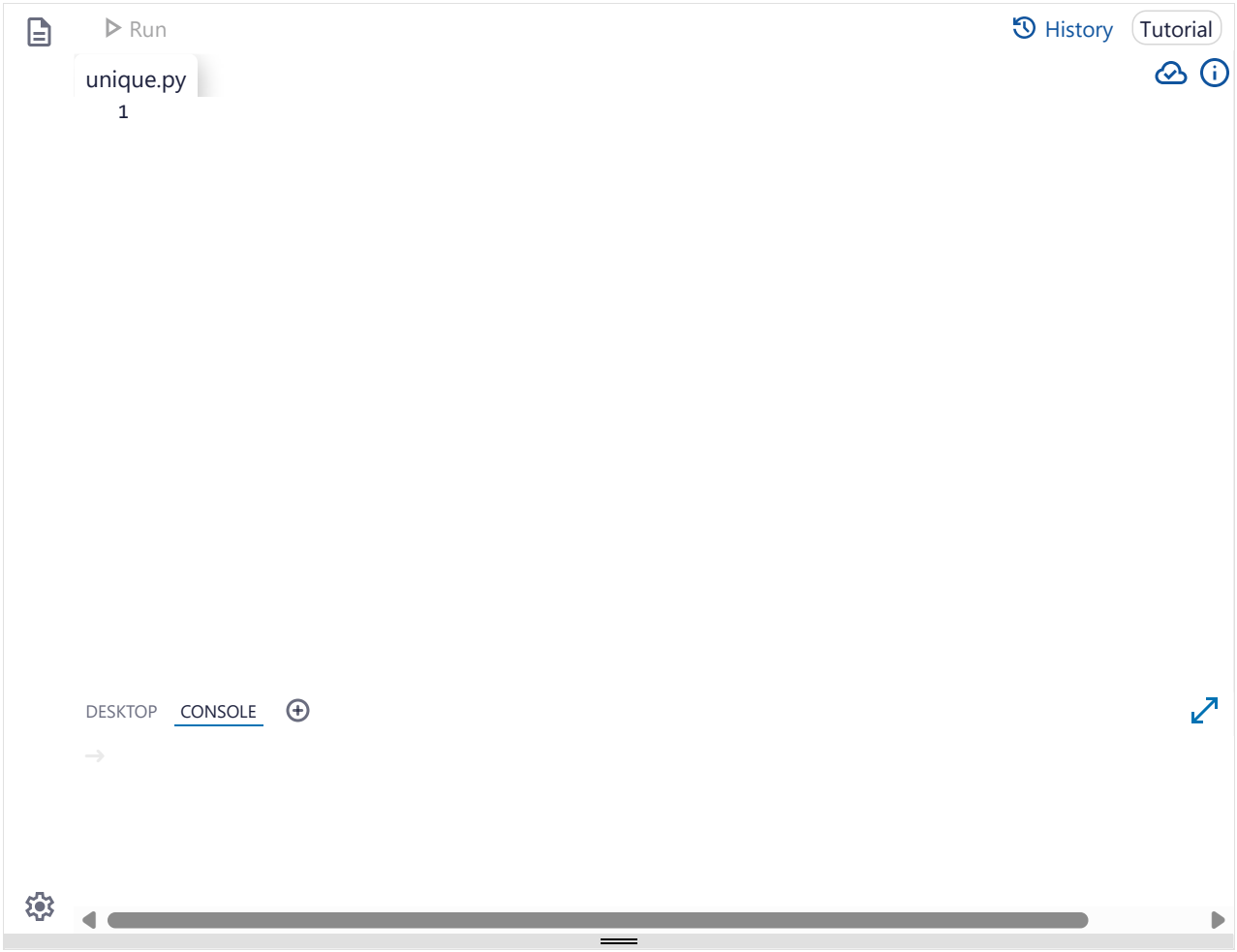
As usual, implement assert tests for your function before submitting it. For example:

- `get_unique_words(["I", "am", "Sam"])` should return `["I", "am", "Sam"]`
- `get_unique_words(["Sam", "Sam", "Sam"])` should return `["Sam"]`

As a fun example, copy this list representation of the full *Green Eggs and Ham* text into your program and verify that there are, indeed, 51 unique words.

If you're looking for a **challenge**, note that a more efficient implementation of this algorithm would use a dictionary rather than a list for unique_words accumulator because finding things in a dictionary is generally faster than finding them in a list. For this alternate implementation, the dictionary key, the word itself, is all you really need so you can assign the dictionary value of an existing word to `None`.

Open new tab | Dock



Submit for grading

Coding trail of your work What is this?

```
History of your effort will appear here once you begin working on this
zyLab.
```

Latest submission

No submissions yet

Template, Model Solution, and Test Bench Show

Lab statistics Show

Student behavior insights (beta) Show

Similarity detection Show

Student activity Show